

Chapter 8 - Practice Set

Dictionaries

Question 1: CRUD Operations on a Contact Dictionary

Write a program that creates a dictionary representing a contact with four fields: name, phone, email, and city. Assign any starting values you like.

- (a) Add a new field called 'country' with any value, and print the full dictionary.
- (b) Read and print the 'email' field using direct key access [].
- (c) Read the field 'fax' using .get() with a default value of 'not available', and print the result.
- (d) Update the 'city' field to a different city and print the full dictionary.
- (e) Remove the 'phone' field using del, then print the dictionary.
- (f) Remove and print the 'email' field using pop(), capturing the returned value in a variable.
- (g) Call popitem() on the remaining dictionary and print the key-value pair that is removed.
- (h) Call clear() and print the now-empty dictionary.

Question 2: Valid and Invalid Dictionary Keys

Write a program that explores which Python types can and cannot be used as dictionary keys.

- (a) Create a single dictionary with five entries, using each of these as a key in turn: a string, an integer, a float, the boolean True, and a tuple of two integers. Assign any values you like. For each entry, print the key, its value, and the type of the key using type().
- (b) Inside a try/except block, attempt to create a new dictionary using a list as a key. Print the exact error message Python produces.
- (c) Demonstrate that a tuple containing the same elements as that list CAN be used as a key successfully: create such a dictionary and print it.

Question 3: len() and Membership Operators

Write a program that creates a dictionary with at least five entries where the keys are school subject names (strings) and the values are integer exam scores.

- (a) Print the total number of subjects using `len()`.
- (b) Choose one subject name that IS in your dictionary. Use `'in'` to confirm it is a key and print a message such as `'math is in the dictionary'`.
- (c) Choose one subject name that is NOT in your dictionary. Use `'not in'` to confirm it is absent and print an appropriate message.
- (d) Pick one score value that IS in your dictionary. Use `.values()` to confirm it is present as a value and print an appropriate message.
- (e) Check whether the score 100 appears in `.values()` and print an appropriate message regardless of whether 100 is in your scores.
- (f) Now check whether one of your subject name strings appears as a VALUE (not a key) using `.values()`, and print the result. This illustrates the difference between checking keys and checking values.

Question 4: Exploring Dictionary Methods

Write a program that creates a dictionary representing a movie with these four keys: title, director, year, and rating. Assign any values you like.

- (a) Print all the keys using `.keys()`.
- (b) Print all the values using `.values()`.
- (c) Use `.items()` in a for loop to print each field and its value on a separate line in the format: `'title: Inception'`.
- (d) Use `setdefault()` to add a `'genre'` field with the value `'Unknown'` only if `'genre'` is not already in the dictionary. Print the dictionary to confirm the result.
- (e) Call `setdefault()` on `'title'` again, passing a different value. Print the dictionary again to confirm that the existing title was NOT overwritten.
- (f) Create a second dictionary with two additional fields of your choice (for example, runtime and language). Use `update()` to merge it into the movie dictionary and print the final result.

Question 5: Looping Through a Dictionary Three Ways

Write a program that creates a dictionary of at least five countries and their capital cities (keys are country names, values are capital cities).

- (a) Loop using `.keys()` and print only the country names, one per line.
- (b) Loop using `.values()` and print only the capital cities, one per line.

(c) Loop using `.items()` with tuple unpacking to print a formatted message for each pair, for example: 'The capital of France is Paris.'

(d) After the three loops, use a dictionary comprehension to create a new dictionary that maps each capital city name to the number of characters in that name. Print this new dictionary.

Question 6: Nested Dictionaries -- Student Records

Write a program that creates a nested dictionary called `'class_records'`. Each key is a student name (string), and each value is another dictionary with exactly three keys: `'subject'` (string), `'score'` (integer), and `'passed'` (boolean: True if score is 60 or above, False otherwise). Include at least three students.

(a) Print the subject and score of one specific student by name using two chained `[]` operators.

(b) Update the score of one student to a new value. Also update their `'passed'` field to match the new score. Print that student's inner dictionary to confirm both changes.

(c) Loop through `class_records` using `.items()` and print only the names of students whose `'passed'` value is True.

(d) Loop through `class_records` and add a `'feedback'` key to each student's inner dictionary. Set it to `'Good work'` if `'passed'` is True, or `'Needs improvement'` otherwise. Print the full nested dictionary after this step.

Question 7: Dictionary with List Values -- Monthly Temperatures

Write a program that creates a dictionary where each key is a month name and the value is a list of at least five daily temperature readings (integers or floats -- you can use made-up numbers). Include at least four months.

(a) Print all temperature readings for one specific month.

(b) Print only the reading at index 2 for that same month.

(c) Add a new temperature reading to one month's list using `.append()` and print that month's updated list.

(d) Use a for loop over `.items()` to calculate and print the average temperature for each month. You may use `sum()` and `len()` to compute each average.

(e) Find and print the single highest temperature reading across all months combined.

Question 8: List of Product Dictionaries

Write a program that creates a list of at least five product dictionaries. Each dictionary must have exactly three keys: 'name' (string), 'category' (string), and 'price' (a number). Use at least two different category values across the five products.

(a) Loop through the list and print the name and price of every product.

(b) Find and print the name and price of the most expensive product.

(c) Create a filtered list containing only products whose price is strictly less than 50. Print the count of such products and the name of each one.

(d) Choose one of your category values. Create a new list containing only the products in that category and print how many products are in it.

Question 9: Word Frequency Counter

Write a program that counts how many times each word appears in a sentence. Use this string as your starting point (or choose your own):

```
text = "the cat sat on the mat and the cat sat"
```

Split the string into individual words using `.split()` and build a frequency dictionary manually using a for loop and the dictionary operations you have learned. Do not use any imported libraries -- build the count dictionary yourself, one word at a time.

(a) Print the complete word frequency dictionary.

(b) Print the total number of unique words using `len()`.

(c) Loop through the dictionary and print only the words that appear more than once.

(d) Find and print the word that appears most often. If there is a tie, printing any one of the tied words is acceptable.

Question 10: Dictionary Comprehension -- Three Scenarios

Write a program that demonstrates dictionary comprehension in three separate scenarios. Write each as its own clearly labelled section.

Scenario A: Given the list

```
languages = ["python", "java", "swift", "rust", "go", "kotlin"]
```

use a single dictionary comprehension to create a dictionary that maps each language name to the number of characters in that name. Print the result.

Scenario B: Given this dictionary of student names and raw scores:

```
raw_scores = {
```

```
"Alice": 92, "Bob": 74, "Carol": 88,
```

"Dave": 61, "Eve": 95, "Frank": 79

}

use a dictionary comprehension to create a new dictionary mapping each student name to a letter grade. Use this scale: 90 or above earns 'A', 80 to 89 earns 'B', 70 to 79 earns 'C', below 70 earns 'F'. Print the result.

Scenario C: Using the same raw_scores dictionary from Scenario B, use a dictionary comprehension with a filter condition to create a new dictionary containing only students who scored 80 or above. Print the result.

End of Chapter 8 Practice Set -- Good luck!